Does Comparing Four Candidate Solutions Improve Answer Selection?

DASE7506 MP2 | Wang Nailin | 25 October 2026

Status. The checkpoint was frozen after validation and scored by the supplied independent evaluator on the complete public test bank: 389/467 (83.30%).

Abstract

We study whether within-question comparison improves selection of a correct final answer from four language-model solutions to a maths problem. Our pre-specified hypothesis was that a ridge scorer using answer agreement, relative answer values, relative text statistics and embedding similarities would outperform an otherwise identical candidate-local scorer. It did not: with the same disagreement-only training groups and ridge penalty, the pointwise model selected 742/798 validation answers correctly (92.98%), versus 724/798 (90.73%) for the relative-feature model. On the 355 validation questions with at least two distinct parsed candidate answers, the respective counts were 299 (84.23%) and 283 (79.72%). Relative features repaired 13 pointwise errors but introduced 31. The result supports the simpler pointwise scorer for this study while leaving other comparative designs open. Its frozen first-version checkpoint scored 389/467 (83.30%) on the complete public test bank. The report distinguishes this test result from the easier full-validation score and discloses subsequent test-score exposure.

1. Task, research question and prior work

For each question, the program sees four candidate solution texts and four supplied 1024-dimensional whole-trace vectors. It returns one index from 0 to 3. The official scorer compares the selected final numeric answer with a reference, accepting equivalent parsed forms; it does not verify every intermediate reasoning step [1]. Multiple candidates may therefore be labelled correct. This is related to selecting solutions with a learned verifier [3] and to answer-consensus selection [4], but our experiments use only the supplied candidates, vectors and train labels, without an extra generator or pretrained model.

Our question is: **Should a candidate be judged on its own, or in relation to the other three?** Before experimentation, we predicted that relative features would improve validation selection, especially when candidate answers disagreed. We also predicted that a simple answer-agreement rule could be useful but would fail on shared mistakes. These hypotheses and the main ablation were recorded before model evaluation in `code/EXPERIMENT_PLAN.md`.

2. Data, supervision and protocol

The provided split contains 4,755 training, 798 validation and 467 test questions, with four candidates per question and fixed embeddings of shape `[N, 4, 1024]` [1,2]. Each candidate's binary training label is one if the supplied parser maps its final answer to the parsed reference answer. All matching candidates are positive. The full training split yields 14,353 positive candidates. Validation was used to develop and select the original checkpoint before its first test score; no test reference was used to fit that checkpoint. Later aggregate test-score exposure before packaging is disclosed in Section 4a. The input-only data checker verified file hashes and alignment.

The test bank contains answer-disagreement questions, whereas 443/798 validation questions have at most one distinct parsed candidate answer. We therefore report both complete-validation accuracy and the **355-question answer-disagreement subset**. This subset is a diagnostic, not an independently sampled test set. After the initial main comparison, we explored training only on the 2,114 training groups with at least two distinct parsed final answers (4,286 positive candidates). This filter uses candidate inputs, not validation/test labels, but its adoption was informed by validation results and is explicitly exploratory.

3. Models and controlled ablation

The fixed-index baseline always chooses candidate 0. The agreement baseline selects a candidate from the largest group of equal parsed final answers, breaking ties by earliest index. Neither uses trained weights.

The **pointwise** scorer uses the supplied 1024-vector plus 12 candidate-local quantities: log text length, word count, number count, equals-sign count and dollar-sign count; presence of a parsed answer, a boxed answer and a numeric-answer tag; negative answer, log answer magnitude and fractional answer flags; and question-word overlap. These features can use the question and one candidate but cannot see competing candidates. For feature vector `x`, train-only mean `mu`, scale `sigma` and weight `w`, the score is `s(x) = ((x-mu)/sigma) dot w + b`. We choose the highest-scoring of four candidates, with first-index tie breaking. We fit ridge least squares to the binary candidate labels, `min_w`

`||Z w - (y-b)||^2 + alpha ||w||^2`, with `b = mean(y)` and `alpha` in `{100, 1000, 10000}`. Standardization parameters are fitted on train only and stored in the checkpoint.

The **relative** scorer retains the same 1024-vector, 12 local features, binary labels, ridge implementation, training groups and alpha. Its only controlled difference is 12 extra within-question features: equal-answer group size and share, uniqueness, whether the answer is the minimum or maximum, log-distance from the median answer, relative text length and number count, mean/maximum/minimum embedding cosine similarity to the other candidates, and distance from the embedding centroid. Removing these 12 features is the key ablation. The comparison is made at matched alpha and matched training population; it does **not** isolate which of the 12 relative features helps or hurts.

After the negative initial result, we also tested two exploratory alternatives: answer-group aggregation of pointwise scores (max/mean plus a small agreement bonus) and a pairwise ridge ranker fitted on 8,271 positive-negative within-question candidate pairs. Neither replaces the controlled feature ablation.

4. Evaluation procedure and results

All validation correctness counts come from the supplied, unchanged `predict.py` and independent `score_predictions.py`. The runner starts a fresh prediction process; a separate scorer process then checks the predictions. A local wall clock covers both processes. We tried the fixed three-value alpha grid, preserved prediction and scorer JSON for each method, and selected the strongest validation checkpoint. The first all-train comparison was pre-specified; the disagreement-only training, aggregation and pairwise experiments followed it and should be interpreted as exploratory validation development.

METHOD	FIT GROUPS	CORRECT / 798	CORRECT / 355	LOCAL TIME, S
Fixed index 0	none	624 (78.20%)	195 (54.93%)	0.50
Answer agreement	none	688 (86.22%)	245 (69.01%)	0.58
Pointwise, alpha 1000	all 4,755	734 (91.98%)	291 (81.97%)	0.58
Relative, alpha 1000	all 4,755	714 (89.47%)	271 (76.34%)	0.70
Pointwise, alpha 100	hard 2,114	**742 (92.98%)**	**299 (84.23%)**	0.62
Relative, alpha 100	hard 2,114	724 (90.73%)	283 (79.72%)	0.71

Table 1. Official validation accuracy and local fresh-process prediction-plus-scoring time. The first two learned rows are the initial all-train controlled comparison; the hard-training rows are an exploratory, again matched, comparison. Times come from two local evaluation runs and are not reference-machine measurements.

At matched alpha 1000 on all training groups, adding relative features reduces correct selections by 20/798 (2.51 percentage points). At matched alpha 100 on disagreement-only training groups, it reduces them by 18/798 (2.26 points). The latter pair also differs on the hard validation subset by 16/355 (4.51 points). For transparency, all-train alpha 100 gives 733/798 pointwise versus 720/798 relative; the full three-alpha grid and scorer files are retained in `code/results/validation/` and `code/results/validation-hard/`. Hard-only pointwise alpha 100 improves over all-train pointwise alpha 100 by 9/798, but this training-population choice was made after inspecting validation and does not establish an independent generalization gain.

EXPLORATORY ALTERNATIVE	BEST SETTING ON VALIDATION	CORRECT / 798	CORRECT / 355
Answer-group score aggregation	max + 0.10 per extra vote	739 (92.61%)	296 (83.38%)
Pairwise ridge preference scorer	alpha 1000	739 (92.61%)	296 (83.38%)

Table 2. Exploratory follow-ups. Both trail the chosen pointwise checkpoint by three questions. A small observed gap after selecting several configurations on the same validation set should not be overinterpreted.

4. Frozen first-version test result

After validation selection, we froze `code/checkpoints-hard/pointwise-ridge-100.npz` and ran the supplied predictor and separate official scorer on all 467 public test questions. The score was **389/467 = 83.29764453961457%**. The test bank consists entirely of answer-disagreement questions; the closer validation diagnostic is **299/355 = 84.23%**, not 742/798 = 92.98%. The test is about 0.93 percentage points below that hard-validation estimate. The official output is `code/results/test-final/score.json`. The checkpoint and settings were unchanged after this first score. Subsequent exploratory branches produced aggregate public-test scores that were seen before this submission package was chosen; we do not present comparisons among those branches as untouched holdout evidence. Their test results were not used to fit this checkpoint.

5. Error analysis and interpretation

For the selected matched hard-training pair, the relative scorer fixes 13 pointwise mistakes but creates 31 new ones; its net loss is 18 questions. At validation position 90, candidate answers are `[48, 48, 32, 44]` with reference `48`. Pointwise chooses `32`, while relative chooses a correct `48`. This is an instance where agreement is helpful. At position 77, answers are `[1120, 1120, 2240, 1120]` with reference `2240`. Pointwise chooses the correct singleton, while relative chooses the incorrect majority. This illustrates how agreement can amplify a shared error. The predictors do not use these positions or question IDs. Of the 31 newly incorrect relative-model choices, 23 switch to a larger equal-answer group; 10 of the 13 corrected choices do so. These paired counts are consistent with greater reliance on agreement, but do not isolate a feature's causal effect. The remaining two net losses occur in low-disagreement validation questions where the relative model chooses an unparsed candidate over the only parsed answer.

Agreement alone beats fixed index by 64 questions overall, but on the 355 disagreement questions it is correct only 245 times, compared with pointwise's 299. The 443 low-disagreement validation questions make whole-bank accuracy look easier: the chosen pointwise checkpoint scores 443/443 there, so its 92.98% whole-bank value should **not** be read as a forecast for the all-disagreement test bank. The harder validation estimate is 299/355 = 84.23%, itself subject to selection bias because variants were chosen on validation.

The evidence rejects our particular relative-feature hypothesis on this bank; it does not show that all candidate comparison is inferior. Potential reasons, not established causal facts, include correlated wrong answers, relative features rewarding agreement too strongly, and a linear additive model's inability to check whether an arithmetic step satisfies the original question. The official label measures final-answer correctness, so neither a correct choice nor a model coefficient proves that reasoning steps were valid. A further controlled ablation separating embeddings, text-format cues, and answer-value features would be useful, but has **not** yet been run and is not claimed here.

6. Resources, reproducibility and limitations

The submitted version inference checkpoint is `code/checkpoints-hard/pointwise-ridge-100.npz`; SHA-256 is `609ca9452a9f37d5130e97de194075c755cb30d64a33c66e9803581b8dcef535`. Its sole required inference asset is 13,972 bytes on disk and 13,248 uncompressed NPZ-member bytes, below 16 MiB. On a Windows 11 AMD Ryzen 9 7940HX PC (32 logical processors), with `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` all set to four, local full-validation prediction plus independent scoring took 0.62 s. The training script reports 2.94 s for the full-train two-model alpha grid and 1.85 s for the hard-train grid;

the pairwise-training script reports 1.55 s. These are local run records, not normalized hardware benchmarks. The complete public-test prediction and independent scoring took 0.5745892 s in two fresh local processes with four-thread environment limits. The 30 s eligibility limit is defined on a Xeon Platinum 8457C; our local AMD measurement cannot certify the reference-machine result.

Python 3.12 and pinned `numpy==2.5.3` are sufficient. From `code/`, after creating and activating a Python 3.12 environment and installing `requirements.txt`, the Windows PowerShell reproduction commands are:

```
$env:OMP_NUM_THREADS='4'; $env:OPENBLAS_NUM_THREADS='4';
$env:MKL_NUM_THREADS='4'
python check_data.py
python train.py --output-dir checkpoints
python train.py --hard-only --output-dir checkpoints-hard
python derive_grouped.py --source checkpoints-hard/pointwise-ridge-
100.npz --output-dir checkpoints-hard
python train_pairwise.py --output-dir checkpoints-hard
python evaluate_validation.py
python evaluate_validation.py --checkpoint-dir checkpoints-hard --
output-dir results/validation-hard
```

For a direct official validation reproduction without retraining, use `python predict.py --method submission.py --checkpoint checkpoints-hard/pointwise-ridge-100.npz --split validation --output runs/validation-predictions.json`, followed by `python score_predictions.py --split validation --predictions runs/validation-predictions.json --output runs/validation-score.json`. The checkpoint path must point to the matching file above. The method uses no random sampling or initialization, so a random seed is not applicable. It was also smoke-tested on a short input slice and an empty bank; it does not require fixed IDs or a fixed bank size. A separate instructor audit-bank result has not been observed.

Reproduction and submission. The repository README gives exact installation, training, validation and test commands. The code, checkpoint, official score JSON and experiment records are included.

AI and reuse disclosure. Substantive AI coding assistance (OpenAI Codex) was used to design the feature sets, implement the training and evaluation scripts. The supplied dataset, parser and evaluation tools were not modified.